

Gestione Stringhe e Manipolazione Caratteri in Assembly 8086

INDICE

1. Gestione delle Stringhe
 2. Manipolazione dei Caratteri
 3. Esempi Pratici
 4. Trucchi e Ottimizzazioni
-

GESTIONE DELLE STRINGHE

Tipi di Stringhe

Stringa C (Null-terminated)

assembly

; Definizione

nome: db 'Mario',0

frase: db 'Ciao mondo!',0

vuota: db 0

; Struttura in memoria:

; nome: 'M' 'a' 'r' 'i' 'o' 0

; byte: 4D 61 72 69 6F 00

Caratteristiche:

- Terminata da byte zero (null terminator)
- Lunghezza variabile
- Standard del linguaggio C
- Per trovare la lunghezza devi scorrere tutta la stringa

Stringa Pascal (Length-prefixed)

assembly

; Definizione

nome: db 5, 'Mario'

frase: db 12, 'Ciao mondo!'

vuota: db 0

; Struttura in memoria:

; nome: 5 'M' 'a' 'r' 'i' 'o'

; byte: 05 4D 61 72 69 6F

Caratteristiche:

- Primo byte = lunghezza
- Lunghezza immediatamente disponibile
- Standard del linguaggio Pascal
- Massimo 255 caratteri (1 byte per lunghezza)

Operazioni Base sulle Stringhe

1. Scorrere una Stringa C

assembly

```
; Metodo 1: Con indice
scorrere_c_indice:
    mov si, 0                ; Indice iniziale
ciclo:
    mov al, [stringa + si] ; Carica carattere
    cmp al, 0                ; Fine stringa?
    je fine                  ; Sì, esci

    ; Elabora carattere in AL
    call elabora_carattere

    inc si                   ; Prossimo carattere
    jmp ciclo
fine:
    ret

; Metodo 2: Con puntatore
scorrere_c_puntatore:
    mov si, offset stringa ; Puntatore iniziale
ciclo:
    mov al, [si]             ; Carica carattere
    cmp al, 0                ; Fine stringa?
    je fine                  ; Sì, esci

    ; Elabora carattere in AL
    call elabora_carattere

    inc si                   ; Prossimo indirizzo
    jmp ciclo
fine:
    ret
```

2. Scorrere una Stringa Pascal

assembly

scorrere_pascal:

```
    mov si, offset stringa
    mov cl, [si]           ; Carica lunghezza
    inc si                 ; Punta al primo carattere

    cmp cl, 0              ; Stringa vuota?
    je fine
```

ciclo:

```
    mov al, [si]           ; Carica carattere

    ; Elabora carattere in AL
    call elabora_carattere

    inc si                 ; Prossimo carattere
    dec cl                 ; Decrementa contatore
    jnz ciclo              ; Continua se non zero
```

fine:

```
    ret
```

3. Calcolare Lunghezza Stringa C

assembly

strlen:

```
    ; Input: SI = puntatore alla stringa
    ; Output: AX = lunghezza
    push si
    mov ax, 0              ; Contatore lunghezza
```

ciclo_len:

```
    cmp byte ptr [si], 0   ; Fine stringa?
    je fine_len
    inc ax                  ; Incrementa contatore
    inc si                  ; Prossimo carattere
    jmp ciclo_len
```

fine_len:

```
    pop si
    ret
```

4. Copiare una Stringa

```

assembly

; Copia stringa C
strcpy:
    ; Input: SI = sorgente, DI = destinazione
    push si
    push di

ciclo_copia:
    mov al, [si]          ; Carica da sorgente
    mov [di], al          ; Salva in destinazione
    cmp al, 0             ; Era il terminatore?
    je fine_copia
    inc si
    inc di
    jmp ciclo_copia

fine_copia:
    pop di
    pop si
    ret

```

MANIPOLAZIONE DEI CARATTERI

Codifica ASCII

Tabella ASCII Essenziale

Carattere	Hex	Dec	Binario
----- ----- ----- -----			
'0'-'9'	30-39	48-57	00110000-00111001
'A'-'Z'	41-5A	65-90	01000001-01011010
'a'-'z'	61-7A	97-122	01100001-01111010
Spazio	20	32	00100000

Differenza Maiuscolo/Minuscolo

'A' = 01000001 (41h)
 'a' = 01100001 (61h)
 Differenza = 00100000 (20h) = bit 5

Conversioni di Caso

Da Minuscolo a Maiuscolo

assembly

to_upper:

; Input: AL = carattere

; Output: AL = carattere maiuscolo

; Verifica se è minuscolo

cmp al, 'a'

jb non_minuscolo ; Minore di 'a'

cmp al, 'z'

ja non_minuscolo ; Maggiore di 'z'

; È minuscolo, converti

and al, 0DFh ; Azzera bit 5 (20h = 00100000b)

non_minuscolo:

ret

; Versione ottimizzata

to_upper_fast:

and al, 0DFh ; Funziona sempre, anche se già maiuscolo

ret

Da Maiuscolo a Minuscolo

assembly

to_lower:

```
; Input: AL = carattere
; Output: AL = carattere minuscolo

; Verifica se è maiuscolo
cmp al, 'A'
jb non_maiuscolo
cmp al, 'Z'
ja non_maiuscolo

; È maiuscolo, converti
or al, 20h          ; Imposta bit 5
```

non_maiuscolo:

ret

; Versione ottimizzata (attenzione: funziona solo con lettere!)

to_lower_fast:

```
or al, 20h          ; Funziona solo se AL è sicuramente una lettera
ret
```

Classificazione Caratteri

Verificare se è una Lettera

assembly

is_letter:

```
; Input: AL = carattere
; Output: Zero Flag set se è lettera

; Converti in maiuscolo per semplificare
push ax
call to_upper

; Verifica se A <= AL <= Z
cmp al, 'A'
jb not_letter
cmp al, 'Z'
ja not_letter

; È una lettera
pop ax
cmp al, al          ; Imposta Zero Flag
ret
```

not_letter:

```
pop ax
cmp al, 0           ; Azzera Zero Flag
ret
```

Verificare se è una Cifra

assembly

```
is_digit:
    ; Input: AL = carattere
    ; Output: Zero Flag set se è cifra

    cmp al, '0'
    jb not_digit
    cmp al, '9'
    ja not_digit

    ; È una cifra
    cmp al, al          ; Imposta Zero Flag
    ret

not_digit:
    cmp al, 0           ; Azzera Zero Flag
    ret
```

Verificare se è una Vocale

assembly

```
is_vowel:
    ; Input: AL = carattere
    ; Output: Zero Flag set se è vocale

    ; Converti in maiuscolo
    push ax
    and al, 0DFh

    ; Controlla tutte le vocali
    cmp al, 'A'
    je found_vowel
    cmp al, 'E'
    je found_vowel
    cmp al, 'I'
    je found_vowel
    cmp al, 'O'
    je found_vowel
    cmp al, 'U'
    je found_vowel

    ; Non è vocale
    pop ax
    cmp al, 0                ; Azzera Zero Flag
    ret

found_vowel:
    pop ax
    cmp al, al                ; Imposta Zero Flag
    ret
```

Conversioni Numeriche

Da Cifra ASCII a Valore Numerico

assembly

ascii_to_digit:

; Input: AL = carattere cifra ('0'-'9')

; Output: AL = valore numerico (0-9)

; Verifica se è cifra

cmp al, '0'

jb invalid_digit

cmp al, '9'

ja invalid_digit

; Converti

sub al, '0' ; '5' - '0' = 5

ret

invalid_digit:

mov al, 0FFh ; Valore di errore

ret

Da Valore Numerico a Cifra ASCII

assembly

digit_to_ascii:

; Input: AL = valore numerico (0-9)

; Output: AL = carattere cifra ('0'-'9')

; Verifica range

cmp al, 9

ja invalid_value

; Converti

add al, '0' ; 5 + '0' = '5'

ret

invalid_value:

mov al, '?' ; Carattere di errore

ret

ESEMPI PRATICI

Esempio 1: Contare le Vocali in una Stringa

assembly

conta_vocali:

; Input: SI = puntatore stringa C

; Output: AX = numero vocali

push si

mov ax, 0 ; Contatore vocali

ciclo_conta:

mov bl, [si] ; Carica carattere

cmp bl, 0 ; Fine stringa?

je fine_conta

; Verifica se è vocale

mov al, bl ; Copia carattere

call is_vowel

jnz non_vocale ; Non è vocale

inc ax ; Incrementa contatore

non_vocale:

inc si ; Prossimo carattere

jmp ciclo_conta

fine_conta:

pop si

ret

Esempio 2: Convertire Stringa in Maiuscolo

assembly

string_to_upper:

; Input: SI = puntatore stringa C
 ; Modifica la stringa originale

push si

ciclo_upper:

mov al, [si] ; Carica carattere
 cmp al, 0 ; Fine stringa?
 je fine_upper

; Converti in maiuscolo

call to_upper

mov [si], al ; Salva carattere convertito

inc si ; Prossimo carattere

jmp ciclo_upper

fine_upper:

pop si

ret

Esempio 3: Confrontare Due Stringhe

assembly

strcmp:

; Input: SI = stringa1, DI = stringa2
; Output: Zero Flag set se uguali

push si
push di

ciclo_confronto:

mov al, [si] ; Carattere stringa1
mov bl, [di] ; Carattere stringa2

cmp al, bl ; Sono uguali?
jne diverse ; No, stringhe diverse

cmp al, 0 ; Fine di entrambe?
je uguali ; Sì, sono uguali

inc si ; Prossimo carattere
inc di
jmp ciclo_confronto

diverse:

pop di
pop si
cmp al, 0 ; Azzera Zero Flag
ret

uguali:

pop di
pop si
cmp al, al ; Imposta Zero Flag
ret

TRUCCHI E OTTIMIZZAZIONI

1. Utilizzo di Tabelle di Lookup

assembly

; Tabella per classificazione caratteri

char_type_table:

db 256 dup(0) ; 0 = altro

; Inizializza tabella in fase di setup

setup_table:

mov si, offset char_type_table

; Marca le lettere

mov cx, 26

mov bx, 'A'

set_letters:

mov [si + bx], 1 ; 1 = lettera

inc bx

loop set_letters

; Ripeti per minuscole...

ret

; Uso della tabella

is_letter_fast:

mov bx, 0

mov bl, al ; Carattere come indice

mov si, offset char_type_table

cmp byte ptr [si + bx], 1

ret ; Zero Flag impostato se lettera

2. Ottimizzazioni Comuni

assembly

```
; Invece di chiamare funzioni per ogni carattere  
; Usa istruzioni in-line per operazioni semplici
```

```
; LENTO:  
call to_upper
```

```
; VELOCE:  
and al, 0DFh
```

```
; LENTO:  
call is_digit  
jz found_digit
```

```
; VELOCE:  
sub al, '0'  
cmp al, 9  
jbe found_digit      ; Below or Equal = unsigned <=
```

3. Gestione Buffer e Stringhe Lunghe

assembly

```
; Per stringhe molto lunghe, usa REP MOVSB
```

```
fast_strcpy:  
    ; Input: SI = source, DI = dest, CX = length  
    cld                      ; Direction flag clear (forward)  
    rep movsb                ; Copia CX byte da SI a DI  
    ret
```

```
; Ricerca carattere veloce
```

```
find_char_fast:  
    ; Input: SI = stringa, AL = carattere, CX = max length  
    cld  
    repne scasb              ; Cerca AL, ferma quando trovato  
    ; DI punta al carattere dopo quello trovato  
    ret
```

4. Gestione Case-Insensitive

assembly

```
strcmp_ignore_case:
    ; Confronto ignorando maiuscolo/minuscolo
    push si
    push di

ciclo_confronto_case:
    mov al, [si]
    mov bl, [di]

    ; Converti entrambi in maiuscolo
    and al, 0DFh
    and bl, 0DFh

    cmp al, bl
    jne diverse

    cmp al, 0
    je uguali

    inc si
    inc di
    jmp ciclo_confronto_case

; ... resto come strcmp normale
```

RIASSUNTO OPERAZIONI COMUNI

Conversioni

- **Maiuscolo:** `AND AL, 0DFh`
- **Minuscolo:** `OR AL, 20h` (solo se già lettera)
- **ASCII → Numero:** `SUB AL, '0'`
- **Numero → ASCII:** `ADD AL, '0'`

Verifiche

- **È lettera:** Converti in maiuscolo, poi `CMP AL, 'A'` e `CMP AL, 'Z'`
- **È cifra:** `CMP AL, '0'` e `CMP AL, '9'`
- **È vocale:** Converti in maiuscolo, confronta con A,E,I,O,U

Stringhe

- **Scorrere C:** Loop fino a byte 0
- **Scorrere Pascal:** Usa primo byte come contatore
- **Copiare:** Loop carattere per carattere
- **Confrontare:** Loop fino a differenza o fine

Questa guida copre tutto quello che serve per manipolare stringhe e caratteri in Assembly 8086!